

lab_10_official

Hadia Beshir

```
library(readr)
db <- read_csv("pdb_stats.csv")
```

Rows: 6 Columns: 9

-- Column specification -----

Delimiter: ","

chr (1): Molecular Type

dbl (8): X-ray, EM, NMR, Integrative, Multiple methods, Neutron, Other, Total

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

>Q1: What percentage of structures in the PDB are solved by X-Ray and Electron Microscopy.

```
total <- sum(db$Total)
(sum(db$`X-ray`) / total) * 100
```

[1] 80.95077

```
(sum(db$EM) / total) * 100
```

[1] 12.83843

According to the data on RCSB, 80.48% of structures in the PDB are solved by X-ray, 13.78% are solved by electron microscopy.

>Q2: What proportion of structures in the PDB are protein?

```
sum(db$Total[1:3]) / total
```

```
[1] 0.979118
```

>Q3: Type HIV in the PDB website search box on the home page and determine how many HIV-1 protease structures are in the current PDB?

After typing HIV in the main search and putting HIV-1 protease, 1227 structures are present in the database.

The PDB Frmat

```
library(bio3d)
pdb <- read.pdb("1HSG.pdb.txt")
print(pdb)
```

```
Call: read.pdb(file = "1HSG.pdb.txt")
```

```
Total Models#: 1
```

```
Total Atoms#: 1686, XYZs#: 5058 Chains#: 2 (values: A B)
```

```
Protein Atoms#: 1514 (residues/Calpha atoms#: 198)
```

```
Nucleic acid Atoms#: 0 (residues/phosphate atoms#: 0)
```

```
Non-protein/nucleic Atoms#: 172 (residues: 128)
```

```
Non-protein/nucleic resid values: [ HOH (127), MK1 (1) ]
```

```
Protein sequence:
```

```
PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYD
QILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFPQITLWQRPLVTIKIGGQLKE
ALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYDQILIEICGHKAIGTVLVGPTP
VNIIGRNLLTQIGCTLNF
```

```
+ attr: atom, xyz, seqres, helix, sheet,
      calpha, remark, call
```

>Q4: Water molecules normally have 3 atoms. Why do we see just one atom per water molecule in this structure?

In protein structures like 1HSG, only one atom, typically an oxygen, for each water molecule is seen. This is likely due to limits in resolution and inability to see hydrogen atoms. This

is because hydrogen has only one electron causing it to be hard to be seen. Hydrogen atoms are also disordered, constantly moving, and disordered particles are not very visible in the final models.

>Q5: There is a critical “conserved” water molecule in the binding site. Can you identify this water molecule? What residue number does this water molecule have

The conserved water molecule in the binding site is known to be flap water. The residue number is usually 301. It is noticed as it acts as a bridge between the flaps of the protease and the ligand.

> Q6: Generate and save a figure clearly showing the two distinct chains of HIV-protease along with the ligand. You might also consider showing the catalytic residues ASP 25 in each chain and the critical water (we recommend “Ball & Stick” for these side-chains). Add this figure to your Quarto document.



Figure 1: “HIV-Pr”



Figure 2: “ASP-25”

Introduction to Bio3D in R

```
library(bio3d)
pdb <- read.pdb("1hsg")
```

Note: Accessing on-line PDB file

```
pdb
```

```
Call: read.pdb(file = "1hsg")
```

```
Total Models#: 1
Total Atoms#: 1686, XYZs#: 5058 Chains#: 2 (values: A B)

Protein Atoms#: 1514 (residues/Calpha atoms#: 198)
Nucleic acid Atoms#: 0 (residues/phosphate atoms#: 0)

Non-protein/nucleic Atoms#: 172 (residues: 128)
Non-protein/nucleic resid values: [ HOH (127), MK1 (1) ]
```

```
Protein sequence:
PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYD
QILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFPQITLWQRPLVTIKIGGQLKE
ALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYDQILIEICGHKAIGTVLVGPTP
VNIIGRNLLTQIGCTLNF
```

```
+ attr: atom, xyz, seqres, helix, sheet,
      calpha, remark, call
```

>Q7: How many amino acid residues are there in this pdb object?

198 amino acid residues are there in this pdb object.

>Q8: Name one of the two non-protein residues?

HOH and MK1 are the two non-protein residues/

>Q9: How many protein chains are in this structure?

2 protein chains are in this structure.

Note that the attributes (+ attr:) of this object are listed on the last couple of lines. To find the attributes of any such object you can use:

```
attributes(pdb)
```

```
$names  
[1] "atom" "xyz" "seqres" "helix" "sheet" "calpha" "remark" "call"
```

```
$class  
[1] "pdb" "sse"
```

To access these individual attributes we use the dollar-attribute name convention that is common with R list objects. For example, to access the atom attribute or component use `pdb$atom`:

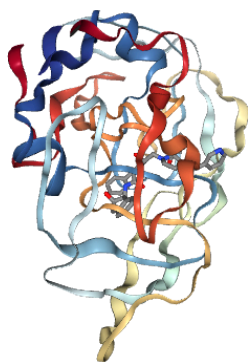
```
head(pdb$atom)
```

	type	eleno	elety	alt	resid	chain	resno	insert	x	y	z	o	b
1	ATOM	1	N	<NA>	PRO	A	1	<NA>	29.361	39.686	5.862	1	38.10
2	ATOM	2	CA	<NA>	PRO	A	1	<NA>	30.307	38.663	5.319	1	40.62
3	ATOM	3	C	<NA>	PRO	A	1	<NA>	29.760	38.071	4.022	1	42.64
4	ATOM	4	O	<NA>	PRO	A	1	<NA>	28.600	38.302	3.676	1	43.40
5	ATOM	5	CB	<NA>	PRO	A	1	<NA>	30.508	37.541	6.342	1	37.87
6	ATOM	6	CG	<NA>	PRO	A	1	<NA>	29.296	37.591	7.162	1	38.40

	segid	elasy	charge
1	<NA>	N	<NA>
2	<NA>	C	<NA>
3	<NA>	C	<NA>
4	<NA>	O	<NA>
5	<NA>	C	<NA>
6	<NA>	C	<NA>

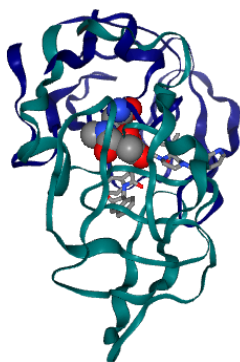
The returned `NGLViewerR` object can be further added to build custom interactive visualizations:

```
library(bio3dview)  
library(NGLViewerR)  
  
view.pdb(pdb) |>  
  setSpin()
```



You can also customize the display in many ways with minimal code. For example, lets custom color the chains and highlight some key residues as spacefill/vdw:

```
sele <- atom.select(pdb, resno=25)
view.pdb(pdb, cols=c("navy","teal"),
         highlight = sele,
         highlight.style = "spacefill") |>
setRock()
```



Predicting functional motions of a single structure

Let's read a new PDB structure of Adenylate Kinase and perform Normal mode analysis.

```
adk <- read.pdb("6s36")
```

```
Note: Accessing on-line PDB file
PDB has ALT records, taking A only, rm.alt=TRUE
```

```
adk
```

```
Call: read.pdb(file = "6s36")
```

```
Total Models#: 1
Total Atoms#: 1898, XYZs#: 5694 Chains#: 1 (values: A)

Protein Atoms#: 1654 (residues/Calpha atoms#: 214)
Nucleic acid Atoms#: 0 (residues/phosphate atoms#: 0)

Non-protein/nucleic Atoms#: 244 (residues: 244)
Non-protein/nucleic resid values: [ CL (3), HOH (238), MG (2), NA (1) ]
```

```
Protein sequence:
MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLRAAVKSGSELGKQAKDIMDAGKLVT
DELVIALVKERIAQEDCRNGFLLDGFPRTPQADAMKEAGINVYVLEFDVPDELIVDKI
VGRRVHAPSGRVYHVKFNPVKVEGKDDVTGEELTTRKDDQEETVRKRLVEYHQMTAPLIG
YYSKEAEAGNTKYAKVDGTPVAEVRADLEKILG
```

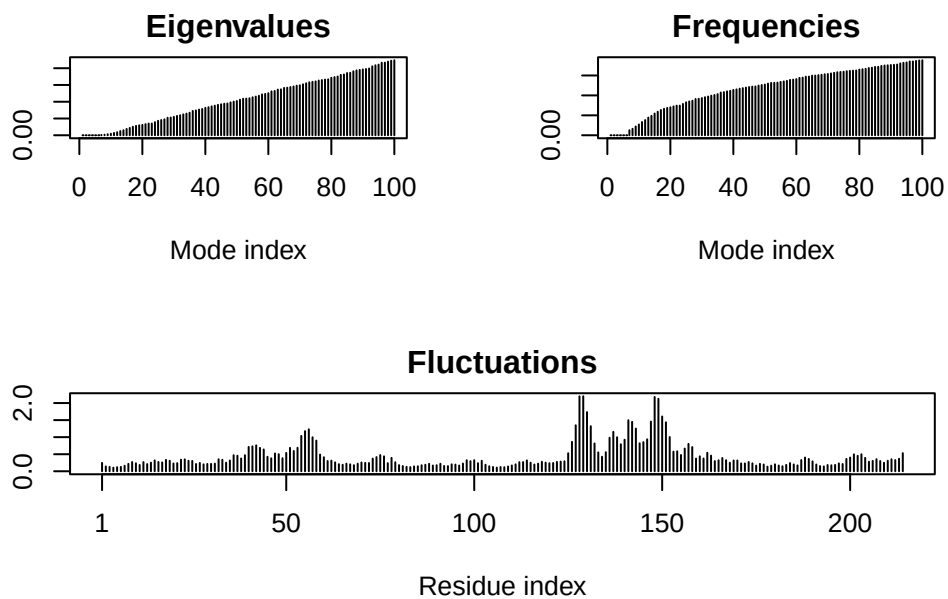
```
+ attr: atom, xyz, seqres, helix, sheet,
      calpha, remark, call
```

Normal mode analysis (NMA) is a structural bioinformatics method to predict protein flexibility and potential functional motions (a.k.a. conformational changes).

```
m <- nma(adk)
```

```
Building Hessian...      Done in 0.025 seconds.
Diagonalizing Hessian... Done in 0.412 seconds.
```

```
plot(m)
```

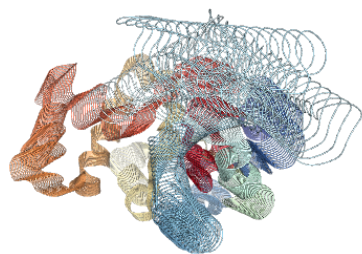


To view a “movie” of these predicted motions we can generate a molecular “trajectory” with the `mktrj()` function.

```
mktrj(m, file="adk_m7.pdb")
```

For a quicker display you can use the `view.nma()` function from the `bio3dview` package mentioned previously:

```
view.nma(m, pdb=adk)
```



>Q10. Which of the packages above is found only on BioConductor and not CRAN?

The packages above is found only on BioConductor and not CRAN is msa instead of the standard.

>Q11. Which of the above packages is not found on BioConductor or CRAN?:

The above packages that is not found on BioConductor or CRAN is bio3dview.

>Q12. True or False? Functions from the pak package can be used to install packages from GitHub and BitBucket?

True, Functions from the pak package can be used to install packages from GitHub and BitBucket.

Search and retrieve ADK structures

```
library(bio3d)
aa <- get.seq("1ake_A")
```

Warning in get.seq("1ake_A"): Removing existing file: seqs.fasta

Fetching... Please wait. Done.

aa

```

      1      .      .      .      .      .      60
pdb|1AKE|A  MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLAAVKSSELGKQAKDIMDAGKLV
      1      .      .      .      .      .      60
      61      .      .      .      .      .      120
pdb|1AKE|A  DELVIALVKERIAQEDCRNGFLLDGFPTIPQADAMKEAGINVDYVLEFDVPDELIVDRI
      61      .      .      .      .      .      120
      121      .      .      .      .      .      180
pdb|1AKE|A  VGRRVHAPSGRVYHVKNPPKVEGKDDVTGEELTRKDDQEETVRKRLVEYHQMTPALIG
      121      .      .      .      .      .      180
      181      .      .      .      214
pdb|1AKE|A  YYSKEAEAGNTKYAKVDGTPVAEVRADLEKILG
      181      .      .      .      214
```

Call:

```

read.fasta(file = outfile)

Class:
  fasta

Alignment dimensions:
  1 sequence rows; 214 position columns (214 non-gap, 0 gap)

+ attr: id, ali, call

```

>Q13. How many amino acids are in this sequence, i.e. how long is this sequence?

```

hits <- NULL
hits$ pdb.id <- c('1AKE_A', '6S36_A', '6RZE_A', '3HPR_A', '1E4V_A', '5EJE_A', '1E4Y_A', '3X2S_A', '6H

```

We can now use function `get.pdb()` and `pdbslit()` to fetch and parse the identified structures

```

files <- get.pdb(hits$ pdb.id, path="pdb", split=TRUE, gzip=TRUE)

```

```

Warning in get.pdb(hits$ pdb.id, path = "pdb", split = TRUE, gzip = TRUE):
pdb/1AKE.pdb.gz exists. Skipping download

```

```

Warning in get.pdb(hits$ pdb.id, path = "pdb", split = TRUE, gzip = TRUE):
pdb/6S36.pdb.gz exists. Skipping download

```

```

Warning in get.pdb(hits$ pdb.id, path = "pdb", split = TRUE, gzip = TRUE):
pdb/6RZE.pdb.gz exists. Skipping download

```

```

Warning in get.pdb(hits$ pdb.id, path = "pdb", split = TRUE, gzip = TRUE):
pdb/3HPR.pdb.gz exists. Skipping download

```

```

Warning in get.pdb(hits$ pdb.id, path = "pdb", split = TRUE, gzip = TRUE):
pdb/1E4V.pdb.gz exists. Skipping download

```

```

Warning in get.pdb(hits$ pdb.id, path = "pdb", split = TRUE, gzip = TRUE):
pdb/5EJE.pdb.gz exists. Skipping download

```

```

Warning in get.pdb(hits$ pdb.id, path = "pdb", split = TRUE, gzip = TRUE):
pdb/1E4Y.pdb.gz exists. Skipping download

```


Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3X2S.pdb.gz exists. Skipping download

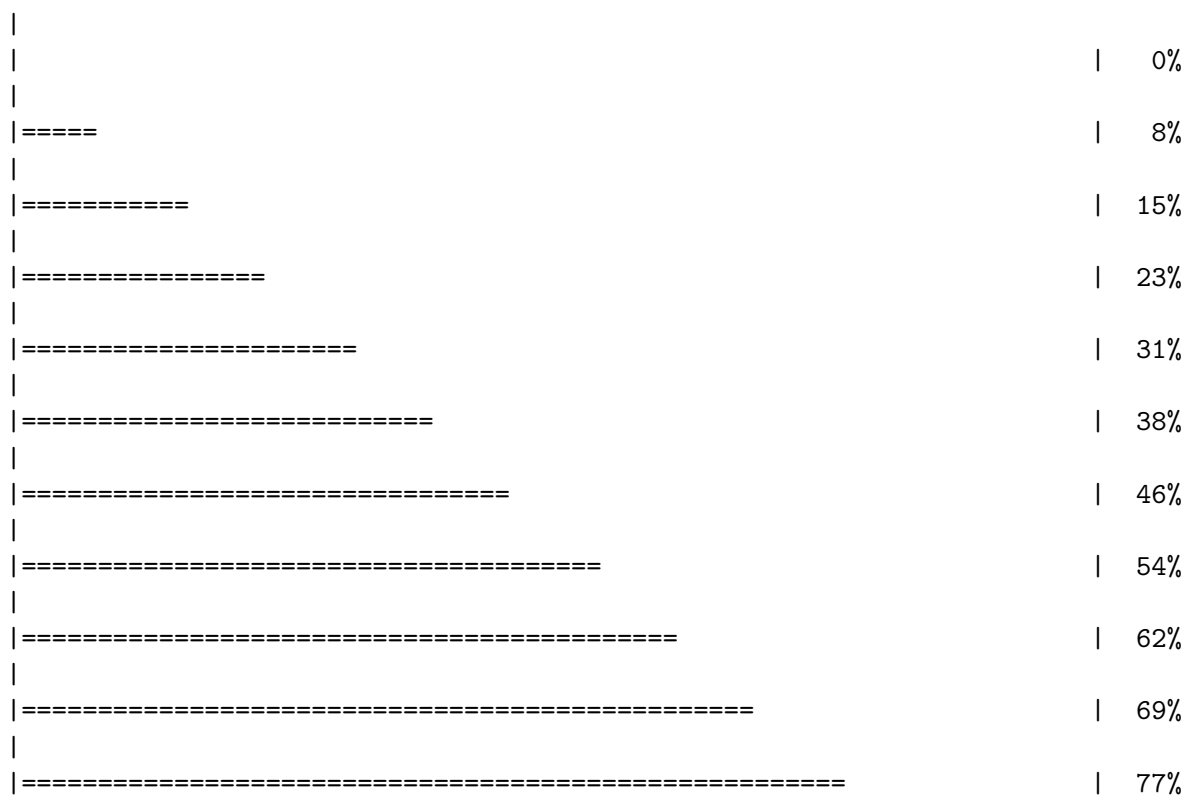
Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAP.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAM.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4K46.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3GMT.pdb.gz exists. Skipping download

Warning in get.pdb(hits\$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4PZL.pdb.gz exists. Skipping download



```

|
|=====| 85%
|
|=====| 92%
|
|=====| 100%

```

Align and superpose structures

Next we will use the `pdbaln()` function to align and also optionally fit (i.e. superpose) the identified PDB structures

```
# Align related PDBs
pdbs <- pdbaln(files, fit = TRUE, exefile="msa")
```

Reading PDB files:

```

pdbs/split_chain/1AKE_A.pdb
pdbs/split_chain/6S36_A.pdb
pdbs/split_chain/6RZE_A.pdb
pdbs/split_chain/3HPR_A.pdb
pdbs/split_chain/1E4V_A.pdb
pdbs/split_chain/5EJE_A.pdb
pdbs/split_chain/1E4Y_A.pdb
pdbs/split_chain/3X2S_A.pdb
pdbs/split_chain/6HAP_A.pdb
pdbs/split_chain/6HAM_A.pdb
pdbs/split_chain/4K46_A.pdb
pdbs/split_chain/3GMT_A.pdb
pdbs/split_chain/4PZL_A.pdb
  PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
..  PDB has ALT records, taking A only, rm.alt=TRUE
.... PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
...

```

Extracting sequences

```

pdb/seq: 1   name: pdbs/split_chain/1AKE_A.pdb
  PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 2   name: pdbs/split_chain/6S36_A.pdb

```

```

PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 3   name: pdbs/split_chain/6RZE_A.pdb
PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 4   name: pdbs/split_chain/3HPR_A.pdb
PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 5   name: pdbs/split_chain/1E4V_A.pdb
pdb/seq: 6   name: pdbs/split_chain/5EJE_A.pdb
PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 7   name: pdbs/split_chain/1E4Y_A.pdb
pdb/seq: 8   name: pdbs/split_chain/3X2S_A.pdb
pdb/seq: 9   name: pdbs/split_chain/6HAP_A.pdb
pdb/seq: 10  name: pdbs/split_chain/6HAM_A.pdb
PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 11  name: pdbs/split_chain/4K46_A.pdb
PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 12  name: pdbs/split_chain/3GMT_A.pdb
pdb/seq: 13  name: pdbs/split_chain/4PZL_A.pdb

```

Annotate collected PDB structures

```

# Vector containing PDB database codes
ids <- basename.pdb(pdb$id)

anno <- pdb.annotate(ids)
unique(anno$source)

```

```

[1] "Escherichia coli"
[2] "Escherichia coli K-12"
[3] "Escherichia coli O139:H28 str. E24377A"
[4] "Escherichia coli str. K-12 substr. MDS42"
[5] "Photobacterium profundum"
[6] "Burkholderia pseudomallei 1710b"
[7] "Francisella tularensis subsp. tularensis SCHU S4"

```

```
anno
```

	structureId	chainId	macromoleculeType	chainLength	experimentalTechnique
1AKE_A	1AKE	A	Protein	214	X-ray
6S36_A	6S36	A	Protein	214	X-ray

6RZE_A ray	6RZE	A	Protein	214	X-
3HPR_A ray	3HPR	A	Protein	214	X-
1E4V_A ray	1E4V	A	Protein	214	X-
5EJE_A ray	5EJE	A	Protein	214	X-
1E4Y_A ray	1E4Y	A	Protein	214	X-
3X2S_A ray	3X2S	A	Protein	214	X-
6HAP_A ray	6HAP	A	Protein	214	X-
6HAM_A ray	6HAM	A	Protein	214	X-
4K46_A ray	4K46	A	Protein	214	X-
3GMT_A ray	3GMT	A	Protein	230	X-
4PZL_A ray	4PZL	A	Protein	242	X-

	resolution	scopDomain	pfam
1AKE_A	2.00	Adenylate kinase	Adenylate kinase, active site lid (ADK_lid)
6S36_A	1.60	<NA>	Adenylate kinase, active site lid (ADK_lid)
6RZE_A	1.69	<NA>	Adenylate kinase, active site lid (ADK_lid)
3HPR_A	2.00	<NA>	Adenylate kinase, active site lid (ADK_lid)
1E4V_A	1.85	Adenylate kinase	Adenylate kinase, active site lid (ADK_lid)
5EJE_A	1.90	<NA>	Adenylate kinase, active site lid (ADK_lid)
1E4Y_A	1.85	Adenylate kinase	Adenylate kinase, active site lid (ADK_lid)
3X2S_A	2.80	<NA>	<NA>
6HAP_A	2.70	<NA>	Adenylate kinase, active site lid (ADK_lid)
6HAM_A	2.55	<NA>	Adenylate kinase, active site lid (ADK_lid)
4K46_A	2.01	<NA>	Adenylate kinase, active site lid (ADK_lid)
3GMT_A	2.10	<NA>	<NA>
4PZL_A	2.10	<NA>	Adenylate kinase (ADK)

	ligandId
1AKE_A	AP5
6S36_A	CL (3),NA,MG (2)
6RZE_A	NA (3),CL (2)
3HPR_A	AP5
1E4V_A	AP5
5EJE_A	AP5,CO

1E4Y_A AP5
 3X2S_A JPY (2),AP5,MG
 6HAP_A AP5
 6HAM_A AP5
 4K46_A ADP,AMP,PO4
 3GMT_A SO4 (2)
 4PZL_A CA,FMT,GOL

ligandName

1AKE_A BIS(ADENOSINE)-5'-
 PENTAPHOSPHATE
 6S36_A CHLORIDE ION (3),SODIUM ION,MAGNESIUM ION (2)
 6RZE_A SODIUM ION (3),CHLORIDE ION (2)
 3HPR_A BIS(ADENOSINE)-5'-
 PENTAPHOSPHATE
 1E4V_A BIS(ADENOSINE)-5'-
 PENTAPHOSPHATE
 5EJE_A BIS(ADENOSINE)-5'-PENTAPHOSPHATE,COBALT (II) ION
 1E4Y_A BIS(ADENOSINE)-5'-
 PENTAPHOSPHATE
 3X2S_A N-(pyren-1-ylmethyl)acetamide (2),BIS(ADENOSINE)-5'-PENTAPHOSPHATE,MAGNESIUM ION
 6HAP_A BIS(ADENOSINE)-5'-
 PENTAPHOSPHATE
 6HAM_A BIS(ADENOSINE)-5'-
 PENTAPHOSPHATE
 4K46_A ADENOSINE-5'-DIPHOSPHATE,ADENOSINE MONOPHOSPHATE,PHOSPHATE ION
 3GMT_A SULFATE ION (2)
 4PZL_A CALCIUM ION,FORMIC ACID,GLYCEROL

source

1AKE_A Escherichia coli
 6S36_A Escherichia coli
 6RZE_A Escherichia coli
 3HPR_A Escherichia coli K-12
 1E4V_A Escherichia coli
 5EJE_A Escherichia coli 0139:H28 str. E24377A
 1E4Y_A Escherichia coli
 3X2S_A Escherichia coli str. K-12 substr. MDS42
 6HAP_A Escherichia coli 0139:H28 str. E24377A
 6HAM_A Escherichia coli K-12
 4K46_A Photobacterium profundum
 3GMT_A Burkholderia pseudomallei 1710b
 4PZL_A Francisella tularensis subsp. tularensis SCHU S4

1AKE_A STRUCTURE OF THE COMPLEX BETWEEN ADENYLATE KINASE FROM ESCHERICHIA COLI AND THE INHIB

6S36_A
 6RZE_A
 3HPR_A
 1E4V_A
 loop
 5EJE_A
 1E4Y_A
 loop
 3X2S_A
 conjugated adenylate kinase
 6HAP_A
 6HAM_A
 4K46_A
 3GMT_A
 4PZL_A

Cryst

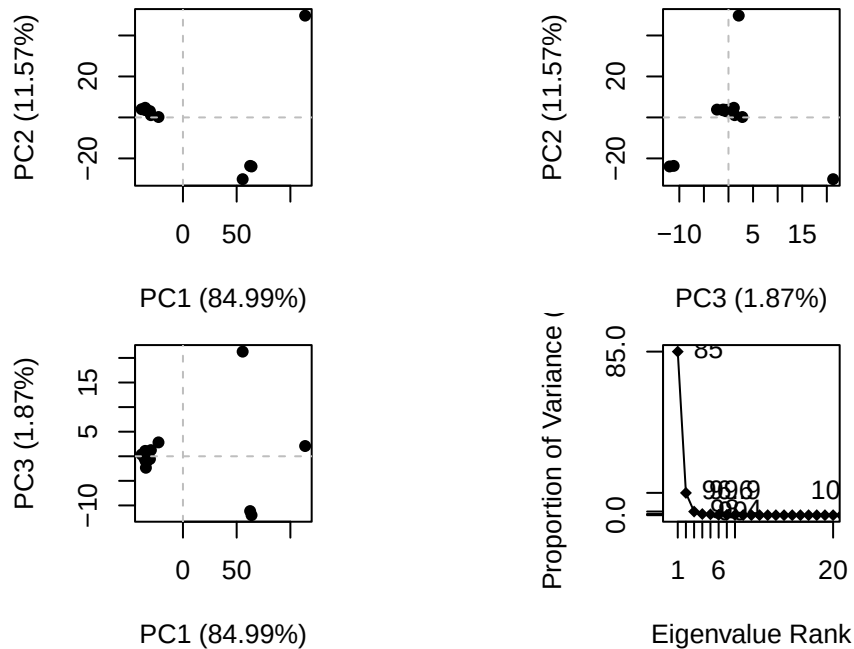
The crys

		citation	rObserved	rFree
1AKE_A	Muller, C.W., et al.	J Mol Biology (1992)	0.19600	NA
6S36_A	Rogne, P., et al.	Biochemistry (2019)	0.16320	0.23560
6RZE_A	Rogne, P., et al.	Biochemistry (2019)	0.18650	0.23500
3HPR_A	Schrank, T.P., et al.	Proc Natl Acad Sci U S A (2009)	0.21000	0.24320
1E4V_A	Muller, C.W., et al.	Proteins (1993)	0.19600	NA
5EJE_A	Kovermann, M., et al.	Proc Natl Acad Sci U S A (2017)	0.18890	0.23580
1E4Y_A	Muller, C.W., et al.	Proteins (1993)	0.17800	NA
3X2S_A	Fujii, A., et al.	Bioconjug Chem (2015)	0.20700	0.25600
6HAP_A	Kantaev, R., et al.	J Phys Chem B (2018)	0.22630	0.27760
6HAM_A	Kantaev, R., et al.	J Phys Chem B (2018)	0.20511	0.24325
4K46_A	Cho, Y.-J., et al.	To be published	0.17000	0.22290
3GMT_A	Buchko, G.W., et al.	Biochem Biophys Res Commun (2010)	0.23800	0.29500
4PZL_A	Tan, K., et al.	To be published	0.19360	0.23680
	rWork	spaceGroup		
1AKE_A	0.19600	P 21 2 21		
6S36_A	0.15940	C 1 2 1		
6RZE_A	0.18190	C 1 2 1		
3HPR_A	0.20620	P 21 21 2		
1E4V_A	0.19600	P 21 2 21		
5EJE_A	0.18630	P 21 2 21		
1E4Y_A	0.17800	P 1 21 1		
3X2S_A	0.20700	P 21 21 21		
6HAP_A	0.22370	I 2 2 2		
6HAM_A	0.20311	P 43		
4K46_A	0.16730	P 21 21 21		
3GMT_A	0.23500	P 1 21 1		
4PZL_A	0.19130	P 32		

Principal component analysis

PCA can be performed on the structural ensemble (stored in the `pdb`s object) with the function `pca.xyz()`, or more simply `pca()`.

```
# Perform PCA
pc.xray <- pca(pdb)
plot(pc.xray)
```



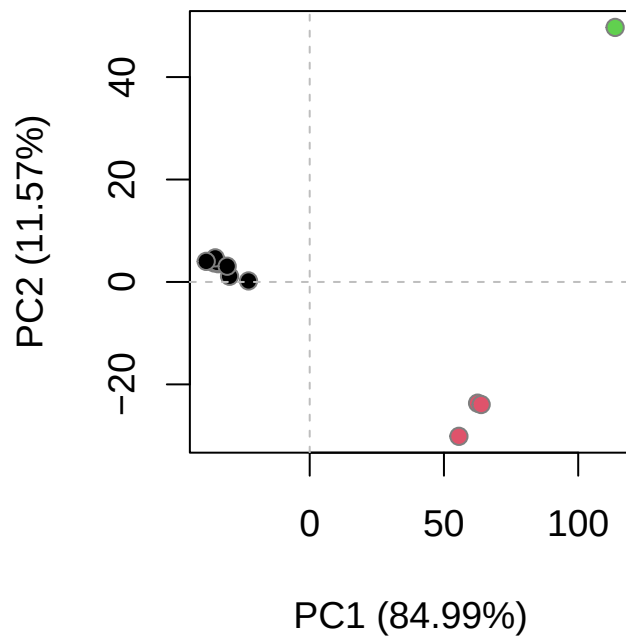
Function `rmsd()` will calculate all pairwise RMSD values of the structural ensemble. This facilitates clustering analysis based on the pairwise structural deviation:

```
# Calculate RMSD
rd <- rmsd(pdb)
```

Warning in `rmsd(pdb)`: No indices provided, using the 204 non NA positions

```
# Structure-based clustering
hc.rd <- hclust(dist(rd))
grps.rd <- cutree(hc.rd, k=3)

plot(pc.xray, 1:2, col="grey50", bg=grps.rd, pch=21, cex=1)
```



PCA visualization

```
# Visualize first principal component
pc1 <- mktrj(pc.xray, pc=1, file="pc_1.pdb")
```

We can also plot our main PCA results with ggplot:

```
#Plotting results with ggplot2
library(ggplot2)
library(ggrepel)
df <- data.frame(PC1=pc.xray$z[,1],
                  PC2=pc.xray$z[,2],
                  col=as.factor(grps.rd),
                  ids=ids)

p <- ggplot(df) +
  aes(PC1, PC2, col=col, label=ids) +
  geom_point(size=2) +
  geom_text_repel(max.overlaps = 20) +
  theme(legend.position = "none")
p
```